Bases de données NoSQL

Qui suis-je?

- 🧑🏾 Nom: Brice FOTZO
- 🌍 Origine: Bafoussam, Cameroun
- 🎓 Formation: Ingénieur Généraliste(BDTN) - ESIGELEC Rouen

Parcours Data & AI

- Analytics Engineer - Apprenti @ Renault
- Data Engineer - CDI @ Starclay
- Tech Lead Data - CDI @ HephIA
- Tech Lead Data - CDI @ Servier
- Intervenant - CDD @ ESIGELEC Rouen/Poitiers

Engagements Communautaires

- Data Engineer - Volunteer @ Validalab(DFG)
- MUG Leader Paris - Volunteer @ MongoDB
- Créateur de contenu @ LinkedIn, Medium
- Créateur de Serial Techos

Et vous?

Faisons connaissance

Rejoindre cet évènement Wooclap



1 Allez sur wooclap.com

2 Entrez le code d'événement dans le bandeau supérieur

Code d'événement
TTFXOB

Activer les réponses par SMS

Plan du module

- 1. Introduction au NoSQL
  - Pourquoi NoSQL & Théorème CAP
  - Les 4 familles NoSQL & cas d'usage
  - Comparaison SQL vs NoSQL
- 2. MongoDB (Orienté Document)
  - Modèle BSON, CRUD & Opérateurs
  - Pipeline d'Agrégation (`$match` , `$group` , `$avg`)
  - Modélisation : *Embedding* vs *Referencing*
- 3. Neo4j (Orienté Graphe)
  - Property Graph Model (Nœuds, Relations, Propriétés)
  - Langage Cypher (`MATCH` , `MERGE` , `collect()` , chemins)
- 4. Architecture Polyglotte & Projets
  - Projet guidé : SmartCity Explorer (FastAPI + Mongo + Neo4j)
  - Projet final : Portfolio d'Ingénieur Polyglotte
- 5. Pour en savoir plus *(en fin de support)*
  - Nouveautés fin 2026 : MongoDB 9.0, IA & Vector Search, GraphRAG, Cypher 25

Organisation du module (3 Jours)

Jour 1 (Lun 09/11 : 9h-12h & 13h30-17h30)

- 09h00 – 11h00 (2h CM) : Panorama NoSQL, MongoDB & Neo4j
- 11h00 – 12h00 (1h TP) : Enrôlement GitHub & Setup Docker `SmartCity`
- 13h30 – 17h30 (4h TP) : Design & Sprints `SmartCity` *(Neo4j Browser & OneCompiler en sandbox)*
- Intersession J1 -> J2 : avancer les 4 Sprints `SmartCity` (`200/200`)

Jour 2 (Lun 16/11 : 9h-12h & 14h-17h — 100% TP)

- 09h00 – 09h30 : Bilan `SmartCity` , questions & pistes de déblocage
- 09h30 – 12h00 : System Design `Portfolio` (`ARCHITECTURE.md` & `API_ROUTES.md`)
- 10h30 : clôture `SmartCity` (dernière chance)
- 14h00 – 15h30 : Soutenances System Design (Jalon 1 — 30%)
- 15h30 – 17h00 : Dev Backend d'équipe & Frontends individuels

Jour 3 (Ven 20/11 : 9h-12h & 14h-17h — 100% TP)

- 09h00 – 12h00 : Intégration finale Backend + 4 Frontends personnels
- 14h00 – 17h00 : Démonstrations Live des Portfolios (Jalon 2 — 70%)

Objectifs d’apprentissage

- Compétences visées :
  - Arbitrer entre modèle relationnel, document (MongoDB) et graphe (Neo4j)
  - Modéliser en NoSQL (*Embedding* vs *Referencing*, Graphe de propriétés)
  - Écrire des pipelines d’agrégation MQL et des traversées Cypher
  - Concevoir une Architecture Polyglotte industrielle en Python
  - *Pour en savoir plus* : usages IA (Vector Search, Voyage AI, MCP, GraphRAG)
- Stack technique du module :
  - Bases NoSQL : `MongoDB 7` et `Neo4j 5` (Docker, TP)
  - Backend & API : `Python 3.11+` , `FastAPI` , `Pydantic`
  - Drivers Async : `Motor` / `PyMongo Async` , `Neo4j Python Driver`
  - Outillage & CI : `Docker Compose` , `uv` , `pytest` , `GitHub Actions`

Introduction au NoSQL

Au-delà du relationnel

Qu’est-ce que le NoSQL ?

Origine

11 juin 2009 à San Francisco

- Meetup organisé par Johan Oskarsson
- Émergence d’un nouveau paradigme

Définitions

- No SQL : N’utilisent pas un modèle relationnel (ni le langage SQL)
- Not only SQL : Bases non relationnelles complémentaires

Caractéristiques principales

- ✅ Open source
- ✅ Conçus pour des clusters puissants
- ✅ Basés sur les besoins du web au 21ème siècle
- ✅ Pas de schéma fixe
- ✅ Permet l’ajout de champs sans contrôles

Évolution historique

1970-2000

Ère des SGBD traditionnels

- Domination des bases relationnelles
- SQL comme standard
- Oracle, MySQL, PostgreSQL

2000-2010

Émergence du NoSQL

- Bulle internet
- Papiers de recherche (Google, Facebook, IBM)
- BDD open source (MongoDB, CouchDB, Neo4j, Redis, Cassandra, HBase, Riak)

2010-Aujourd'hui

Ère du Cloud DBaaS

- MongoDB Atlas
- AWS DynamoDB
- Google Cloud Firestore
- Azure Cosmos DB
- Cloudant

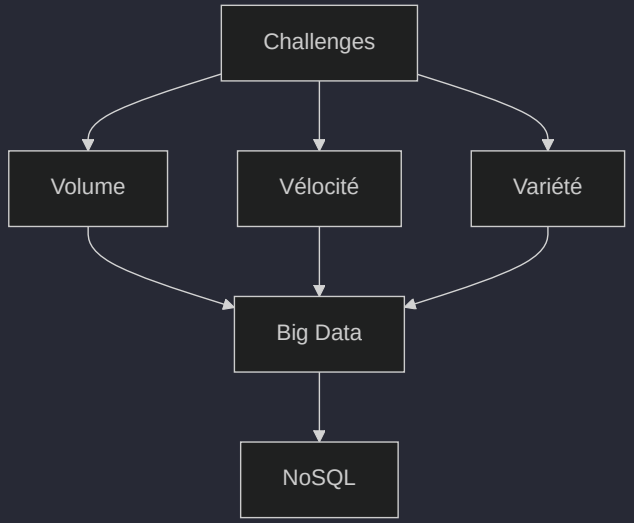
Pourquoi NoSQL ?

Limites du SQL

- Schéma rigide
- Scalabilité verticale
- Performances avec grands volumes
- Structures complexes
- Coût des jointures

Besoins modernes

- Données non structurées
- Scalabilité horizontale
- Haute disponibilité
- Temps réel
- Agilité de développement



Le théorème CAP

Cohérence

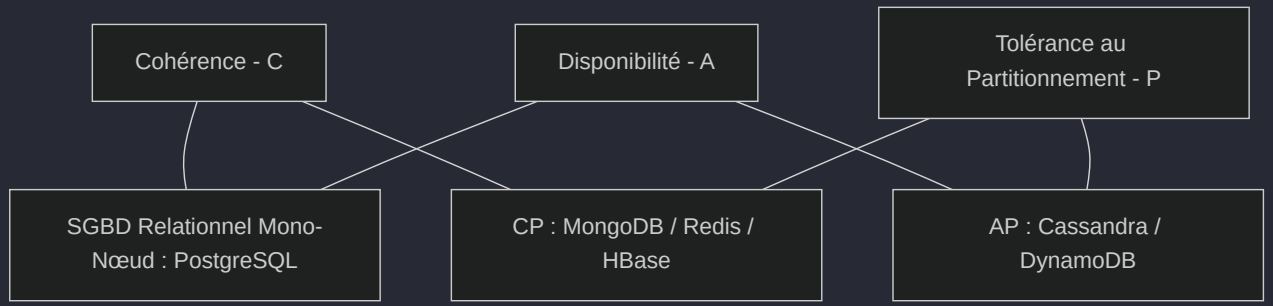
- Tous les nœuds voient les mêmes données
- Lectures cohérentes
- Transactions ACID

Disponibilité

- Chaque requête reçoit une réponse
- Sans erreur
- Sans garantie de fraîcheur

Tolérance au partitionnement

- Le système continue de fonctionner
- Malgré les pertes de messages
- Ou pannes réseau



Types de bases NoSQL

Document (MongoDB)

```
{
  "id": 1,
  "nom": "Dupont",
  "adresses": [
    {
      "type": "domicile",
      "ville": "Paris"
    },
    {
      "type": "travail",
      "ville": "Lyon"
    }
  ]
}
```

Colonnes (Cassandra)

```
CREATE TABLE users (
  user_id uuid,
  year int,
  month int,
  events map<timestamp, text>,
  PRIMARY KEY ((user_id), year, month)
);
```

Graphe (Neo4j)

```
CREATE (u:User {name: 'Dupont'})
CREATE (p:Post {title: 'NoSQL'})
CREATE (u)-[:AUTHORED]→(p)
```

Clé-valeur (Redis)

```
SET user:1:nom "Dupont"
SET user:1:email "dupont@mail.com"
HSET user:1 nom "Dupont" email "dupont@mail.com"
```

Caractéristiques des BDD NoSQL

🌐 Communautés impliquées et actives

- Proviennent de l'Open Source ou ont une version Open Source
- Support des communautés fondamental pour la croissance
- Exemples :
  - MongoDB : 858 contributeurs
  - Cassandra : 472 contributeurs
  - Redis : 775 contributeurs

📊 Passage à l'échelle (Scalabilité)

Scalabilité Horizontale ✅

- Ajout de serveurs de même capacité
- Ex: 8 serveurs 2vCPUs/4Go
- Approche privilégiée par NoSQL

Scalabilité Verticale ❌

- Augmentation de la puissance d'un serveur
- Plus coûteuse et limitée

✅ Disponibilité garantie

- Serveur unique : Point de défaillance unique
- Cluster de serveurs : Réplication des données
- Haute disponibilité native

☁️ Cloud Ready et coûts réduits

- Architectures Cloud natives
- Paradigme Cloud intégré
- Serveurs standards, open source
- Implémentation peu coûteuse

Exemple de coût :

- MongoDB Community Edition : 0$
- Oracle RAC : $47,500 par unité

⚡ Hautes performances

- ⏱️ Réponse rapide
- 👥 Haute concurrence
- Nombreux utilisateurs simultanés

🔄 Flexibilité

- Schéma flexible : Modification à la volée

Comparaison SQL vs NoSQL

SQL

- Schéma fixe
- Relations
- Transactions ACID
- Jointures
- Scalabilité verticale
- Requêtes complexes
- Cohérence forte

NoSQL

- Schéma flexible
- Documents imbriqués
- BASE (Basic Availability)
- Dénormalisation
- Scalabilité horizontale
- Requêtes simples
- Cohérence éventuelle

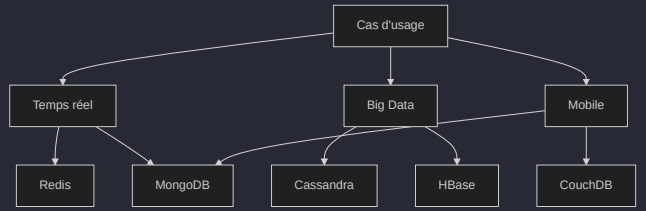
Critères de choix

1. Nature des données (structurées vs non structurées)
2. Volume et croissance prévue
3. Patterns d'accès (lectures vs écritures)
4. Besoins en cohérence
5. Expertise de l'équipe

Cas d’usage

Applications idéales

- Réseaux sociaux
- IoT / Big Data
- Logging
- E-commerce
- Temps réel
- Contenu mobile



Exemples concrets

- Facebook : Cassandra
- LinkedIn : MongoDB
- Twitter : Redis
- Amazon : DynamoDB

Cas d’utilisation de MongoDB

Idéal pour

- Applications web modernes
- Contenu varié et évolutif
- Données semi-structurées
- Besoin de scalabilité horizontale
- Développement agile
- Applications temps réel

Exemples concrets

1. E-commerce
   - Catalogues de produits
   - Paniers utilisateurs
   - Historique des commandes
2. Applications mobiles
   - Profils utilisateurs
   - Contenu généré par les utilisateurs
   - Données de géolocalisation
3. Analytics
   - Logs d’événements
   - Données IoT
   - Métriques en temps réel

Cas d’utilisation de Neo4j

Idéal pour

- Réseaux sociaux
- Systèmes de recommandation
- Détection de fraude
- Graphes de connaissances
- Gestion des identités
- Analyse d’impact

Exemples concrets

1. Réseaux sociaux

```
// Trouver les amis des amis
MATCH (p:Personne {nom:"Alice"})-[:CONNAIT]→(ami)-[:CONNAIT]→(amiDami)
WHERE NOT (p)-[:CONNAIT]→(amiDami)
RETURN DISTINCT amiDami.nom as suggestions
```

2. Recommandations

```
// Produits fréquemment achetés ensemble
MATCH (p:Produit)←[:A_ACHETE]-(c:Client)-[:A_ACHETE]→(autreProduit:Produit)
WHERE p.id = "123"
RETURN autreProduit.nom, count(*) as frequence
ORDER BY frequence DESC
LIMIT 5
```

3. Détection de fraude

```
// Identifier les cycles de transactions suspects
MATCH chemin = (c:Compte)-[:TRANSFERT*3..5]→(c)
```

Testons nous un peu

Connectez-vous à ce wooclap



MongoDB

Une base de données orientée document

Qu’est-ce que MongoDB ?

Caractéristiques principales

- Base de données NoSQL orientée document
- Documents au format BSON (Binary JSON)
- Schéma flexible et dynamique
- Scalabilité horizontale native
- Haute disponibilité avec la réplication

Architecture

- Collections ≈ Tables en SQL
- Documents ≈ Lignes en SQL
- Champs ≈ Colonnes en SQL
- Index supportés
- Pas de jointures natives

Structure des données

Document BSON

```
{
  "_id": ObjectId("5f7d3"),
  "nom": "Dupont",
  "age": 30,
  "adresses": [
    {
      "type": "domicile",
      "ville": "Paris",
      "code_postal": "75001"
    },
    {
      "type": "travail",
      "ville": "Lyon",
      "code_postal": "69001"
    }
  ],
  "tags": ["VIP", "Premium"]
}
```

Types de données

- Strings
- Numbers (Integer, Float, Decimal)
- Dates
- Boolean
- Arrays
- Embedded Documents
- ObjectId
- Null
- Binary Data
- Regular Expression

Opérations CRUD

Create

```
// Insérer un document
db.clients.insertOne({
  nom: "Dupont",
  age: 30
})

// Insérer plusieurs documents
db.clients.insertMany([
  { nom: "Martin", age: 25 },
  { nom: "Bernard", age: 35 }
])
```

Update

```
// Modifier un document
db.clients.updateOne(
  { nom: "Dupont" },
  { $set: { age: 31 } }
)

// Modifier plusieurs documents
db.clients.updateMany(
  { age: { $lt: 30 } },
  { $set: { statut: "jeune" } }
)
```

Read

```
// Trouver tous les clients
db.clients.find()

// Avec filtre
db.clients.find({
  age: { $gt: 30 }
})
```

Delete

```
// Supprimer un document
db.clients.deleteOne({
  nom: "Dupont"
})

// Supprimer plusieurs documents
db.clients.deleteMany({
  age: { $lt: 25 }
})
```

Requêtes avancées & Mises à jour riches

Opérateurs de comparaison & regex

```
// ⩾ 100 000 hab., en Normandie ou Bretagne,
// nom commençant par "rou" (insensible à la casse)
db.cities.find({
  population: { $gte: 100000 },
  region: { $in: ["Normandie", "Bretagne"] },
  name: { $regex: "^rou", $options: "i" }
})
  .sort({ overall_score: -1 })
  .skip(0)
  .limit(20)
```

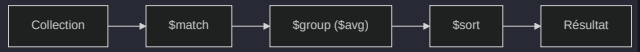
Mises à jour atomiques ($set, $inc, $push)

```
db.cities.updateOne(
  { id: 1 },
  {
    $set: { region: "Normandie" },
    $inc: { population: 500 },
    $push: { scores: { category: "sport", score: 8.5 } }
  }
)
```

Pipeline d'Agrégation MongoDB (`aggregate`)

Principe : traiter les documents par étapes (*stages*)

1. `$match` : Filtrer les documents en entrée *(utilise les index !)*
2. `$group` : Regrouper et calculer (`$avg` , `$sum` , `$min` , `$max`)
3. `$sort` / `$limit` : Trier et limiter les résultats
4. `$project` : Formater les champs en sortie



Exemple concret (utilisé dans `SmartCity`)

Calculer la note moyenne et le nombre d'avis par ville :

```
pipeline = [
    # 1. Filtrer les avis d'une ville
    {"$match": {"city_id": city_id}},
    # 2. Calculer la moyenne des notes
    {
        "$group": {
            "_id": "$city_id",
            "avg_rating": {"$avg": "$rating"},
            "total_reviews": {"$sum": 1}
        }
    }
]
results = await db["reviews"].aggregate(pipeline).to_list(1)
```

Indexation avec MongoDB

Types d'index

- Single Field Index
- Compound Index
- Text Index
- Geospatial Index
- ...

Bonnes pratiques

- Index sur les champs fréquemment recherchés
- Attention aux index composés : ordre important
- Éviter la sur-indexation
- Utiliser explain() pour analyser
- Index en arrière-plan pour la production

Création d'index

```
// Index simple
db.clients.createIndex({ nom: 1 })

// Index composé
db.clients.createIndex(
  { ville: 1, age: -1 }
)

// Index unique
db.clients.createIndex(
  { email: 1 },
  { unique: true }
)
```

Analyse des index

```
// Analyser une requête
db.clients.find({
  ville: "Paris"
}).explain("executionStats")

// Liste des index
db.clients.getIndexes()

// Statistiques d'index
db.clients.stats()
```

Modélisation : Embedding vs Referencing

1. Embedding (Imbrication dans le même document)

```
{
  "id": 1,
  "name": "Rouen",
  "scores": [
    { "category": "transports", "score": 8.2 },
    { "category": "cadre_de_vie", "score": 7.9 }
  ]
}
```

2. Referencing (Collection séparée avec clé)

```
{
  "_id": "65f1a...",
  "city_id": 1,
  "author": "Alice",
  "rating": 5,
  "comment": "Super ville étudiante !"
}
```

La Règle d'Or d'Arbitrage (Query-First Design)

- ✅ Embedding (`1:1` ou `1:N` borné) :
  - Les sous-éléments sont peu nombreux (ex: les 8 scores d'une ville, les expériences/formations d'un CV).
  - Ils sont toujours lus en même temps que le document parent (1 seule lecture disque, zéro jointure).
- ✅ Referencing (`1:N` non borné ou `N:M`) :
  - Le nombre d'éléments grossit indéfiniment (ex: des milliers d'avis `reviews` sur une ville).
  - Ou l'entité est partagée / paginée indépendamment (ex: `projets` communs à plusieurs étudiants).
  - *Rappel :* Limite d'un document BSON = 16 Mo (interdiction des tableaux non bornés !).

Testons nous un peu

Connectez-vous à ce wooclap

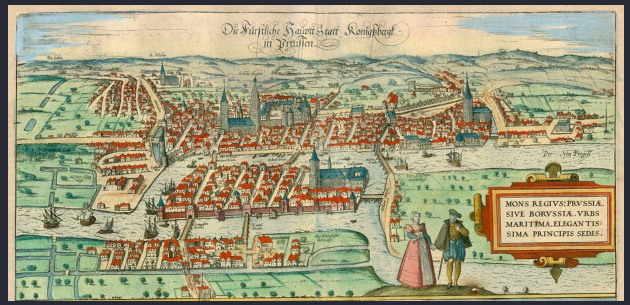


Neo4j

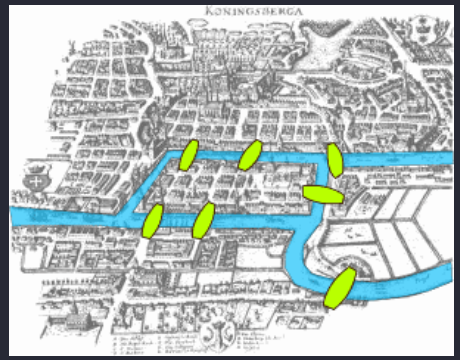
Base de données orientée graphe

Le problème des 7 ponts

Ponts de Koenigsberg



Vue simplifiée



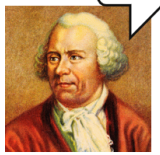
Le problème des 7 ponts

Penser graphe!

Les 7 ponts de Königsberg

HephIA

Peut on faire le tour de la ville en empruntant tous les ponts sans jamais en un prendre un plus d'une fois?



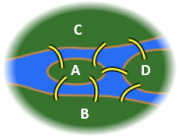
Leonhard Euler
(1701- 1783)

Simplifions le problème à l'extrême!

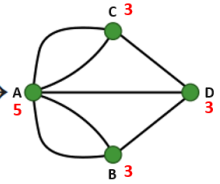
Réponse :







4 zones à visiter



Surfaces terrestres => Sommets
Ponts => arêtes

Démontré par
Carl Hierrholzer
(1840 - 1871)

- Un graphe connexe admet un **parcours eulérien** si et seulement si ses sommets sont tous de **degré pair** *sauf au plus deux*.
- Un graphe connexe admet un **circuit eulérien** si et seulement si tous ses sommets sont de **degré pair**.

Qu’est-ce que Neo4j ?

Caractéristiques principales

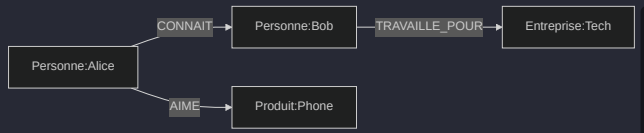
- Base de données orientée graphe
- Modélisation naturelle des relations
- Langage de requête Cypher
- ACID compliant
- Très performant pour les données connectées

Architecture

- Nœuds (Nodes) ≈ Entités
- Relations (Relationships) ≈ Connexions
- Propriétés ≈ Attributs
- Labels ≈ Types de nœuds
- Types de relations

Structure d’un graphe

Éléments de base



Types de nœuds (Labels)

- Personne
- Entreprise
- Produit
- Lieu
- Événement

Relations et propriétés

```
CREATE (p:Personne {
  nom: "Alice",
  age: 30
})-[r:TRAVAILLE_POUR {
  depuis: "2020",
  poste: "Développeur"
}]→(e:Entreprise {
  nom: "Tech Corp",
  secteur: "IT"
})
```

Langage Cypher (CRUD & Idempotence)

Création (`CREATE` vs `MERGE`)

```
// CREATE : crée toujours (risque de doublon)
CREATE (p:Personne {nom: "Alice", age: 30})

// MERGE : "get or create" sans doublon (idéal pour les seeds !)
MERGE (c:City {city_id: 1})
SET c.name = "Rouen", c.overall_score = 8.1

// Relier deux nœuds existants sans doublon
MATCH (c:City {city_id: 1}), (cr:Criterion {name: "Transports"})
MERGE (c)-[:STRONG_IN]→(cr)
```

Lecture (`MATCH` + `WHERE`)

```
MATCH (c:City)-[:STRONG_IN]→(cr:Criterion)
WHERE c.city_id = 1
RETURN cr.name AS point_fort ORDER BY point_fort
```

Modification

```
// Mettre à jour un nœud
MATCH (p:Personne {nom: "Alice"})
SET p.age = 31

// Modifier une relation
MATCH ()-[r:TRAVAILLE_POUR]→()
SET r.poste = "Senior Dev"
```

Suppression

```
// Supprimer un nœud
MATCH (p:Personne {nom: "Alice"})
DELETE p

// Supprimer une relation
MATCH ()-[r:CONNAIT]→()
DELETE r
```

Requêtes Cypher Avancées (`WITH`, `collect`, Chemins)

1. Chemins de longueur variable & Plus court chemin

```
// Remonter toute une chaîne d'actionnariat (1 à N sauts)
MATCH (m:Media {nom: "Le Monde"})<-[:OWNS*]-(owner)
WHERE NOT (owner)<-[:OWNS]-()
RETURN DISTINCT owner.nom
```

2. Chaînage `OPTIONAL MATCH`, `WITH` et agrégation `collect()` (utilisé dans `SmartCity`)

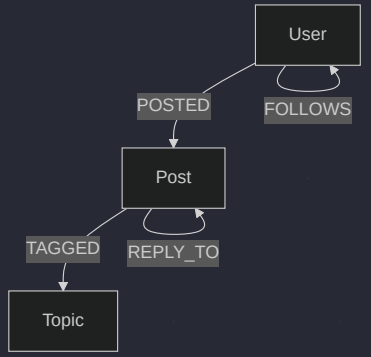
```
// Trouver les K villes similaires ET collecter leurs points forts communs en une seule requête
MATCH (source:City {city_id: $city_id})-[r:SIMILAR_TO]->(target:City)
OPTIONAL MATCH (source)-[:STRONG_IN]->(cr:Criterion)<-[:STRONG_IN]-(target)
WITH target, r.score AS similarity_score, collect(cr.name) AS common_strengths
ORDER BY similarity_score DESC
LIMIT $k
RETURN properties(target) AS city, similarity_score, common_strengths
```

Modélisation avec Neo4j

Bonnes pratiques

1. Labels
   - Utilisez des labels multiples
   - Hiérarchie de labels
   - Nommage clair
2. Relations
   - Direction significative
   - Nommage explicite
   - Propriétés utiles
3. Propriétés
   - Attributs atomiques
   - Indexation stratégique
   - Pas de tableaux complexes

Patterns communs



Anti-patterns

- Relations sans direction
- Propriétés redondantes
- Sur-utilisation des labels
- Relations génériques

Indexation et Performances

Types d'index

- Indexes par label/propriété
- Composite indexes
- Full-text indexes
- Range indexes

Création d'index

```
// Index simple
CREATE INDEX FOR (p:Personne)
ON (p.nom)

// Index composé
CREATE INDEX FOR (p:Personne)
ON (p.nom, p.age)

// Index full-text
CREATE FULLTEXT INDEX personneSearch
FOR (p:Personne)
ON EACH [p.nom, p.description]
```

Optimisation

1. Stratégies d'indexation
   - Index sur les propriétés filtrées
   - Index sur les propriétés de jointure
   - Éviter la sur-indexation
2. Analyse des performances

```
PROFILE
MATCH (p:Personne)-[:CONNAIT]→()
RETURN p.nom, count(*) as relations
```

3. Monitoring
   - Utilisation de la mémoire
   - Temps de réponse
   - Cache hits/misses